

Name: Manav Uttekar

Roll no:71

Problem Statement: Implement the Heap/Shell sort algorithm implemented in Java demonstrating heap/shell data structure with modularity of programming language.

```
import java.io.*;
import java.util.*;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;

class ExternalSortSimple {
    static final int CHUNK_SIZE = 5;

    public static void main(String[] args) throws Exception {
        List<File> chunks = sortChunks("input.txt");
        mergeChunks(chunks, "output.txt");
        System.out.println("Sorted data saved to output.txt");
    }

    static List<File> sortChunks(String fileName) throws IOException {
        List<File> chunks = new ArrayList<>();
        BufferedReader br = new BufferedReader(new FileReader(fileName));
        String line;
        List<Integer> buffer = new ArrayList<>();

        while ((line = br.readLine()) != null) {
            buffer.add(Integer.parseInt(line));
            if (buffer.size() == CHUNK_SIZE) {
                chunks.add(writeChunk(buffer));
                buffer.clear();
            }
        }
        if (!buffer.isEmpty()) chunks.add(writeChunk(buffer));
        br.close();
        return chunks;
    }

    static File writeChunk(List<Integer> data) throws IOException {
        Collections.sort(data);
        File temp = File.createTempFile("chunk", ".txt");
        temp.deleteOnExit();
        BufferedWriter bw = new BufferedWriter(new FileWriter(temp));
        for (int n : data) bw.write(n + "\n");
        bw.close();
        return temp;
    }

    static void mergeChunks(List<File> chunks, String outputFile) throws
    IOException {
        while (chunks.size() > 1) {
            List<File> newChunks = new ArrayList<>();
```

```

        for (int i = 0; i < chunks.size(); i += 2) {
            if (i + 1 < chunks.size()) {
                newChunks.add(merge(chunks.get(i), chunks.get(i +
1)));
            } else newChunks.add(chunks.get(i));
        }
        chunks = newChunks;
    }
    Files.copy(chunks.get(0).toPath(), new File(outputFile).toPath(),
java.nio.file.StandardCopyOption.REPLACE_EXISTING);
}

```

```

static File merge(File f1, File f2) throws IOException {
    BufferedReader r1 = new BufferedReader(new FileReader(f1));
    BufferedReader r2 = new BufferedReader(new FileReader(f2));
    File out = File.createTempFile("merged", ".txt");
    out.deleteOnExit();
    BufferedWriter w = new BufferedWriter(new FileWriter(out));

    String a = r1.readLine(), b = r2.readLine();
    while (a != null && b != null) {
        if (Integer.parseInt(a) < Integer.parseInt(b)) {
            w.write(a + "\n"); a = r1.readLine();
        } else {
            w.write(b + "\n"); b = r2.readLine();
        }
    }
    while (a != null) { w.write(a + "\n"); a = r1.readLine(); }
    while (b != null) { w.write(b + "\n"); b = r2.readLine(); }

    r1.close(); r2.close(); w.close();
    return out;
}
}

```

OUTPUT:

PS C:\Users\Manav\Desktop\exp of DSA> 5

>> 1

>> 9

>> 3

>> 7

>> 6

>> 2

>> 8

>> 4

>> 0

>> \$ javac ExternalSortSimple.java

>> \$ java ExternalSortSimple

>> Sorted data saved to output.txt

>>

>> \$ cat output.txt

>> 0

>> 1

>> 2

>> 3

>> 4

>> 5

>> 6

>> 7

>> 8

>> 9

>> █